

Introduction to Structured Programming KU

Session 0: Organization

Tatjana Hofer

Slides by Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security

Introduction

(To Introduction to Structured Programming)

General Information



Summer Semester Team

- **Practicals Only!**
- Lecture is helpful, but **not** a formal prerequisite.
- We explain contents specifically for the practical assignments.

Winter Semester Lecture

- Run by a different team.
- Recordings are available on YouTube .
- Separate exam dates.

General Information

Differences to Winter

Unlike the main Winter course, we have:

- ✗ **No** weekly exercises.
- ✗ **No** coding sessions or homeworks.
- ⚠ **No course website!**

(We only use specific links, e.g., for the Style Guide. Check Moodle/Emails for everything else.)

i Goal: This track is for students repeating the practicals or taking them off-season.

Course Organization: The Team



Lecturers

Univ.-Prof. Dr. Alexander Felfernig
Dr. techn. Trang Tran
Viet-Man Le, MSc

*"Responsible for the course content,
lectures, and exams."*

>_ KU Sessions

Sebastian David Albu



Assignment Tutorials

Corresponding Tutor
(Check your group registration!)

Teaching Assistants (TAs)

Your personal guides through the semester.

Group	Teaching Assistant
1	Tatjana Hofer
2	Sebastian David Albu
3	Klemens Hofmann
4	Martin Gamper

Group	Teaching Assistant
5	Margarita Dubina
6	Lenon Osmani
7	Marco Kutscha
8	Lukas Miklautz



Contact Policy:

Your TA is your **First Point of Contact** for issues.

(Find mail addresses in the TeachCenter)

Communication Channels

 Main Channel: Discord

Join Server 

<https://discord.gg/nyHEPK3VV4>

Use this for:

- `</>` Assignments & Code help
-  Technical support (VM setup, ...)
-  General organisation: All announcements will be done via Discord!

"There is no such thing as a stupid question..."
...but please try to google it first! 😊

Communication Channels

Email Policy

Strictly for PERSONAL questions only! (e.g., sickness, grades, sensitive issues).

Who to contact?

 **Your Tutor**
(First point of contact)

 **Course Org**
Tatjana Hofer

 **Lecturers**
Dr. Trang Tran

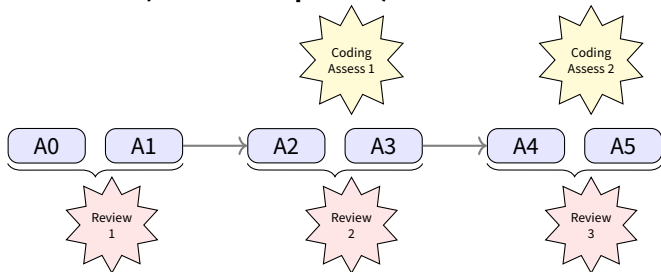
Course Structure

</> The Core

5 (+1) Assignments (A0 – A5) in C.

- **Individual Work Only!** (Plagiarism checks active)
- Course is held in presence (Theory/Tutorials recorded).

❗ Mandatory Attendance (The Checkpoints)



Optional Attendance (Recommended)

Check TUGonline!

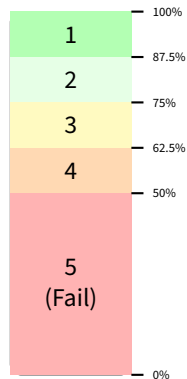
Rooms may change. TUGonline is the single source of truth.

16:00 – 18:00 Thursday	 Theory (Held by Sebastian) & Tutorials (Held by other tutors) <i>One each per assignment (Except A5)</i> 📍 HS i8
↓	
18:00 – 19:00 Thursday	 Question Hours for the corresponding assignments <i>Open floor for issues - Bring your devices so we can debug together!</i> 📍 LS IX

**Course Introduction today is also held in the Theory slot.*

Grading Scale

Grade	Percentage
1	$\geq 87.5\%$
2	$\geq 75.0\%$
3	$\geq 62.5\%$
4	$\geq 50.0\%$
5	$< 50.0\%$



Grading Criteria

The Basics

- Submission of assignments **A1 – A5**.
- You receive points based on the correctness of your code.

Mandatory Requirements (Fail Conditions)

If you miss these, you fail or lose all points for an assignment:

-  **A0 is Mandatory:** No submission = No Grade (Unranked).
-  **A5 Threshold:** You must pass at least **50%** of public test cases.
-  **Assignment Reviews:** **No Show = 0 points** for the reviewed assignment(s).
-  **Coding Assessments:** **No Show = 0 points** for the corresponding assignment (A3 or A5).

Assignment Point Overview

Assgn.	Pts	Related Lectures (Click to Watch ▶)
A0	1	Practicals 1
A1	9	Lecture 2 , Lecture 3
A2	15	Lecture 4 , Lecture 5 , Lecture 6
A3	15	Lecture 7 , Lecture 8 , Lecture 9
A4	25	Lecture 10
A5	35	<i>(Synthesis of all previous concepts)</i>
Sum	100	

📌 Note: These recordings are from the Winter Semester.
They cover the exact theory required for the respective assignment.
However, we also have the theoretical KU sessions which (interactively) cover the same content!

Schedule: Phase 1 (March)

Date	Event	Room
05.03.	A0 Theory	HS i8
06.03.	A1 Theory	HS i2
12.03.	A1 Tutorial	HS i8
12.03.	A0 Question Hour	LS IX
15.03.	Course Registration Deadline	–
18.03. (18:00)	A0 Deadline	–
19.03.	A2 Theory	HS i8
19.03.	A0 + A1 Question Hour	LS IX
25.03. (18:00)	A1 Deadline	–

Schedule: Phase 2 (April)

Date	Event	Room
26.03.	A2 Tutorial	HS i8
13.04. – 16.04.	A0 + A1 Interviews	LS I
16.04.	A3 Theory	HS i8
16.04.	A2 Question Hour	LS IX
22.04. (18:00)	A2 Deadline	–
23.04.	A3 Tutorial	HS i8
30.04.	A4 Theory	HS i8
30.04.	A3 Question Hour	LS IX

Schedule: Phase 3 (May)

Date	Event	Room
06.05. (18:00)	A3 Deadline	–
07.05.	A4 Tutorial	HS i8
11.05. – 13.05.	A2 + A3 Interviews	LS VI / FSI 2
13.05.	A4 Question Hour	LS IX
18.05. (15-17h)	Coding Assessment 1	(Check Mail)
20.05. (18:00)	A4 Deadline	–
21.05.	A5 Release	–
27.05. (15-17h)	Backup Coding Assessment 1	i3, LS II

Schedule: Phase 4 (June – July)

Date	Event	Room
28.05.	A5 Tutorial	HS i8
11.06.	A5 Question Hour	LS IX
17.06. (18:00)	A5 Deadline	–
22.06. – 26.06.	A4 + A5 Interviews	LS II
29.06. (15-17h)	Coding Assessment 2	(Check Mail)
06.07. (15-17h)	Coding Assess. Replacement	HS i13, LS I
07.07.	Second Chance Interview (A5)	LS IX

i Note: Coding Assessment rooms are split across:
HS i2, HS i11, LS II, LS IV, LS IX.
Check TUGonline, Discord or Email before the exam!

The Tutorials: From Theory to Practice

Concept Recap (By Sebastian)

We review the important concepts from the lecture, but **much briefer**.
(Focus is on application, not derivation.)

Live Coding

- **Live Programming Examples:** Watch us write code in real-time.
- **Concrete Tips:** Specific strategies for solving the current assignment.

Pro Tip:

The tutorials often contain code snippets that are *very similar* to what you need for the assignment!
Coding (or at least thinking) along with us is recommended!

Assessment of Assignments

The Procedure

- **Same rules** as in the Winter Semester practicals.
- Detailed criteria are linked in the **TeachCenter**.



The Grader

Who grades my code?

- Your specific Group Tutor is responsible for your grades.
- Please contact **them** first regarding points or feedback!

Assignment Grading Scheme

 [Click here for detailed Assessment Guidelines](#)

70% Functionality

30% Quality

Functionality (70%)

Based on automated test cases:

- **Public Tests:** You see these immediately.
- **Private Tests:** Hidden until grading.

Code Quality (30%)

Manual review by Tutors:

- Style Guide compliance.
- Clean code structure.
- General guidelines.

Assignment Grading Scheme

Deductions (The "Point Eaters")

Points can be deducted (even to negative) for:

- **Rule Violations:** Ignoring specs (e.g., using banned libraries).
- **Memory Issues:** Leaks or invalid access (valgrind errors).
- **Coding Assessment:** Failing the interview leads to deductions/0 points.

Assignment Reviews (Interviews)

! Mandatory Attendance

Held in presence. Missing a review results in **0 points** for the respective assignments.

When?

- Review 1: After **A0 + A1**
- Review 2: After **A2 + A3**
- Review 3: After **A4 + A5**

 **Duration:** $\approx$ 20 minutes (total for the block)

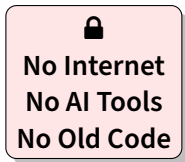
The Agenda

- **Feedback:** Discussion on your submission.
- **Theory Check:** Questions about the concepts used.
- **Code Explanation:** Explain *why* you wrote the code that way.

Coding Assessments

The Format

- **Mandatory** after A3 and A5.
- Held in presence using the **Safe Exam Browser (SEB)**.
- **Task:** Solve small tasks by writing *new* tiny programs (from scratch).
- **Duration:** $\approx$ 90 minutes.

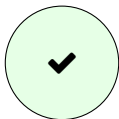


Grading Consequences

Your points for **A3** and **A5** are conditional!

- Points will be **adjusted (deducted)** based on your performance here.
- This ensures that you actually understand the code you submitted at home.

Tools and Materials



Voluntary Practice

 Enhance your understanding

- **What is it?** A questionnaire tool with theory questions.
- **Attempts:** Unlimited! 
- **Goal:** Purely for practice and better understanding.

"A great way to prepare for the Assignment Review theory questions!"

Materials and Tools

Discord

- Announcements (Check regularly!)
- Questions & Support
- Memes ;)

TeachCenter

- General course information
- Lecture slides & other materials

KnowledgeCheckR

Git and GitLab

The most important tool!

- Getting Assignments
- **Submissions** (Push to Master)

Virtual Machine

We provide a standardized VM.

- Pre-configured with all tools.
- *Highly recommended* to avoid OS issues.

The Virtual Machine (VM)



System Specs

- **OS:** Linux, Ubuntu 24.04.
- **Same VM** as in the Winter Semester.
- Download link in the **TeachCenter**.



The Golden Rule

The VM is the Reference Machine for grading.

- C behavior depends on the OS.
- **GitLab Pipeline results are final!**
- If it fails in the pipeline, it fails the assignment.

The Virtual Machine (VM)

Usage

- Requires **VirtualBox** (or VMWare/UTM).
- **Highly Recommended** for beginners!
- All tools pre-installed (Compiler, Valgrind, etc.).
- Everything works *out-of-the-box*.

 *"But it works on my machine!"*
...is **not** a valid excuse.
Always check the Pipeline!

Plagiarism

Plagiarism: Zero Tolerance

🚫 The Rules

We have a zero-tolerance policy!

- **Individual Work Only:** No team work allowed.
- **Do not copy code** (from colleagues or old repos).
- **Protect your code:** Do not show it to others. It is your responsibility to keep your repository private.

🔍 Detection

- Automated checks run at the end of the semester.
- **Every commit counts**, not just the final submission!

Plagiarism: Zero Tolerance



The Source (Provider)

If you let someone copy:

0 Points
on the assignment.

The Plagiarist (Copier)

If you copy:

Graded with "Täuschungsversuch"!
(Academic Dishonesty)

Plagiarism: The Good News (Citations)

The 20% Rule

Up to **20%** of your code may be cited from external sources (Books, Internet, Colleagues, AI) **IF** correctly cited.

How to cite (must include Start/End markers):

```
1 // START CITATION
2 // Source: ChatGPT 4.0
3 // Prompt: "How to reverse a string in C?"
4 // Link: https://chat.openai.com/share/unique-link-id
5
6 char* reverse(char* s)
7 {
8     // ... code ...
9 }
10 // END CITATION
11
```

Plagiarism: The Good News (Citations)



- **Students:** If you share a snippet, **BOTH** must cite each other!
- **AI Tools:** Must provide Model, Prompt, and Permanent Link.

Submissions

Submission Process

git The System: GitLab

- We use **Git** and **GitLab**.
- You will receive your personal repository by:
 **11.03.2025** (hopefully earlier)

i Don't worry: You don't need the repository to start **A0**. You can write the code locally and push it later!

The Rules

- 1 Upload to GitLab:** Pushing code is the only valid submission method.
- 2 One File Only:** Unless specified otherwise, usually only one `.c` file is allowed.
- 3 Read the Description:** Check exactly which file to submit!

⊘ We do NOT accept other modes of submission!

No Email  No USB Sticks  No Discord Messages 

The Mentoring Program

Who are they?

- Students from higher semesters who already passed ISP.
- Each mentor leads a group of ≈ 10 students.

What can they help with?

- `</>` Coding in general
-  Computer setups
-  General study questions

Join the Program

Check your email for the form!

- Takes ≈ 5 minutes.
- Form is for *preferences* only.
- **You will be assigned a mentor in any case.**

Deadline:

XX.03.2026, 23:59

The Mentoring Program

Important Distinction

Mentors are **NOT** Teaching Assistants!

- They help you survive the university start.
- They **do not** grade your assignments.

(You'll have your mentor assigned by Friday!)

Helpful Tips

Tips and Helpful Remarks

Reality Check

This course requires **high effort!**

- **Start Early:** The assignments (especially A4/A5) are larger than they look.
- Procrastination is the number one reason for failing this course.

Tips and Helpful Remarks

The Foundation

Essential for **CS, SEM, ICE** majors. You need this for:

- Object-Oriented Programming 1 (OOP1)
- Computer Org. & Networks (CON)
- System Level Programming (SLP)
- Operating Systems (OS)
- ...and many more!

The Philosophy

"You don't learn programming from a book."

Practice is everything!

Write Code → Break Code → Fix Code.

 **Don't hesitate to ask questions if anything is unclear!**

Motivation

Better said: Why even bother with C?

The C Programming Language

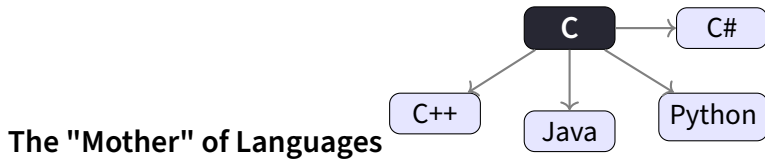
🕒 The Origins (1970)

- **Bell Labs:** Dennis Ritchie & Ken Thompson.
- **Imperative:** Step-by-step instructions.
- **Abstraction:** A thin layer over Assembly.
- *"Programming close to the hardware."*

❓ Why do we teach C?

- **Simplicity:** Only 32 Keywords (C99).
- **Low Abstraction:** You learn how the machine actually works.
- **Foundations:**
 - Pointers & Memory Management
 - Compilers & Makefiles

The C Programming Language



Simplicity: Only 32 Keywords (C99)

"If you know these 32 words, you know the entire syntax vocabulary of C."

☰ The Vocabulary

auto	double	int	struct	break	else	long	switch
case	enum	register	typedef	char	extern	return	union
const	float	short	unsigned	continue	for	signed	void
default	goto	sizeof	volatile	do	if	static	while

✓ Compare this to C++ (>90 keywords) or Java (>50 keywords)!

Characteristics of C

Efficiency & Optimization

- **High Performance:** Very efficient runtime.
- **Mature:** Highly optimized compilers (GCC, Clang) over 50+ years.
- **De-facto Standard:** The king of Systems Programming.

Portability vs. Independence

High Portability: Implementations exist for almost *any* system (Supercomputers to Toasters).

⚠ BUT: Not Platform-Independent!

- Unlike Java, you must recompile for every architecture.
- `int` size may vary between systems!

What is programmed in C?

"C is the bedrock of modern computing."

Operating Systems

- Linux Kernel, Windows, MacOS...
- Embedded Systems (IoT)

>_ System Tools

- **Shells:** sh, bash, zsh
- **GNU Coreutils:** ls, cp, rm, mkdir, cat...

</> Other Languages

- **Java:** The JVM is written in C/C++.
- **Python:** The standard interpreter (CPython) is C.

Libraries & Drivers

- Graphics drivers, Networking stacks, Database engines...

Time for a Break?

During the Break maybe set up your device

Tasks (Setup your Device)

1. Virtual Machine (VM)

Notion: Setup your Device

- Download the VM software and the image.
- *Note: Just download for now, no complex setup needed yet.*

2. GitLab Login

Go to gitlab.tugraz.at

- Click **SSO Login**.
- Use your standard TUGonline credentials.

Tasks (Setup your Device)

3. Eduroam (WiFi)

If you haven't set it up yet:

- **User:** <tug-user>@tugraz.at (NOT your email!)
- **Password:** Separate Eduroam password (set in TUGonline).
- **Tool:** Download config tool at eduroam.tugraz.at.

Break Over

Let us continue with the Setup

Step 1: Virtual Machine (VM)

Installation Steps

- 1 Download:** Get the correct VM image and software (VirtualBox/UTM) for your OS.
- 2 Import:** Import the file into the software.
 - *Tip: Often works via Drag & Drop!*
- 3 Configure:** Adapt settings if necessary.
 - **RAM:** Increase if you have enough (e.g., 4GB or 8GB).
- 4 Start:** Boot up the VM.

i Note: Inside the VM, all tools (Git, GCC, etc.) are pre-installed.
You work *inside* this window.

Step 2: GitLab SSH Keys



1. Generate the Key

```
1 # Navigate to the .ssh directory
2 cd ~/.ssh
3
4 # Generate the key (Press Enter for all prompts)
5 ssh-keygen -o -t rsa -b 4096
6
```

Step 2: GitLab SSH Keys

2. Copy the Public Key

- **Private Key** (`id_ssh`): **NEVER share this!**
- **Public Key** (`id_ssh.pub`): This goes to GitLab.

```
1 # Print the key to the terminal
2 cat id_ssh.pub
3
4 # Copy the output using Ctrl+Shift+C (Terminal shortcut!)
5
```

3. Add to GitLab: Settings → SSH Keys → Paste.

Step 3: Git Configuration

⚙️ One-Time Setup

Run these commands inside the VM terminal:

```
1 # 1. Set your Name
2 git config --global user.name "Your Name"
3 # IMPORTANT: Keep the quotation marks!
4
5 # 2. Set your Email
6 git config --global user.email "mail@student.tugraz.at"
7 # IMPORTANT: Use your TU Graz email!
8
```

Step 3: Git Configuration



Clone the Repository

```
1 git clone <repo-url>
2 # Replace <repo-url> with your link from GitLab
3 # (Remove the < > brackets!)
4
```

Step 4: The Workflow (Submission)

How to submit your solution:

```
1 # 1. Create/Edit a file
2 touch file.txt                # Create empty file
3 mousepad file.txt            # Edit with editor
4
5 # 2. Stage the file (Prepare for commit)
6 git add file.txt
7
8 # 3. Commit (Save snapshot locally)
9 git commit -m "Added solution for A1"
10 # Always include a meaningful message in quotes!
11
12 # 4. Push (Upload to GitLab)
13 git push
```

Step 4: The Workflow (Submission)



✔ That's it!

To submit an assignment, just **add**, **commit**, and **push**.
Repeat this process whenever you change your code.

Hello World

Your First C Program



```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello, World");
6     return 0;
7 }
```

 **Library:** Contains the `printf` function.

 **Entry Point:** Every C program starts in `main`.

 **Action:** Prints text to the terminal.

 **Exit:** Status code.
0 means "Success".

Compiling: Turning Code into Programs



>_ The Basic Way

```
1 gcc test.c           # Creates a.out
2 ./a.out              # Runs the program
3
```

Compiling: Turning Code into Programs

The Professional Way (Flags)

```
1 gcc -Wall -Wextra -g -o test test.c
```

2

- `-o test`: Name the output file "test" (instead of a.out).
- `-Wall -Wextra`: Enable **Warnings** (Vital!).
- `-g`: Add debug info (for Valgrind/GDB).

Warning Policy

Treat warnings like errors! If your submission throws warnings, we deduct points.

The Makefile: Your Best Friend

Typing long `gcc` commands is tedious. We provide a **Makefile**.

Commands

- `make bin:`
Just compile.
- `make run:`
Compile and run immediately.

Grading Standard

We use the Makefile to grade you.

- It enforces our compiler flags.
- Ensure your code compiles cleanly with `make`!

Feel free to open the Makefile and look inside. It helps you understand how C projects are built!

The Testrunner

We provide an automated test system to check your code against our requirements.

1 Download the testrunner (check TeachCenter).

2 Run the tests:

```
1 make test
```

```
2
```

3 View the report:

```
1 make test && firefox testreport.html
```

```
2
```

✔ **Goal:** Get all bars to turn **GREEN!**

Questions?

Introduction to Structured Programming KU

Session 0: Organization

Tatjana Hofer

Slides by Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security